

Block 2: Object-Oriented Programming in Python

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 8

Thread Handling in Python

ഒരു program-നുള്ളിൽ ഒരേ സമയം പല ജോലികൾ ചെയ്യാൻ thread-കൾ എങ്ങനെ ഉപയോഗിക്കാം എന്നാണ് ഈ unit പഠിപ്പിക്കുന്നത്. threading module ഉപയോഗിച്ച് thread ഉണ്ടാക്കുന്നതും join, daemon thread, Lock, RLock, Condition, Semaphore, Queue, thread pool, Timer എന്നിവയും real-world ഉദാഹരണങ്ങളും ഇതിൽ ഉണ്ട്.

Threads and processes

Benefits and challenges of threads

Thread lifecycle

The Global Interpreter Lock (GIL) and thread use cases

Creating and starting a thread

Thread names and identifiers

Useful thread functions

The join() method

Daemon and non-daemon threads

Thread synchronization and race conditions

Locks (mutexes) and deadlock

Reentrant locks (RLock)

Condition variables

Semaphores

Thread pools (concurrent.futures)

Thread communication with Queue

Timer threads

Multithreading: real-world examples

Threads and processes

Process: an instance of a running program, with its own memory space and system resources.

Thread: the smallest unit of execution within a process; threads of a process share its memory.

Multithreading: running several threads concurrently within a single process.

Process: സ്വന്തമായി memory space-ഉം system resources-ഉം ഉള്ള, run ചെയ്തുകൊണ്ടിരിക്കുന്ന ഒരു program-ന്റെ instance.

Thread: ഒരു process-നുള്ളിലെ execution-ന്റെ ഏറ്റവും ചെറിയ unit; ഒരു process-ലെ thread-കൾ അതിന്റെ memory പങ്കിടുന്നു. **Multithreading:** ഒരൊറ്റ process-നുള്ളിൽ പല thread-കൾ concurrent ആയി run ചെയ്യുന്നത്.

When a Python program starts, the interpreter process begins with one thread, the **main thread**. More threads can be created to run code concurrently with it. Each thread has its own stack (execution context) and register values, but all threads of a process share the same memory (global variables, heap, code). Sharing makes communication fast, but two threads changing the same data at the same time can corrupt it, like two people editing the same cell of a shared spreadsheet.

മലയാളത്തിൽ

ഒരു hotel അടുക്കള ഒരു process ആണെന്ന് കരുതുക. അതിലെ ഓരോ cook-ഉം ഒരു thread. എല്ലാവരും ഒരേ അടുക്കളയും ഒരേ ഫ്രിഡ്ജും (shared memory) ആണ് ഉപയോഗിക്കുന്നത്, പക്ഷേ ഓരോരുത്തർക്കും സ്വന്തം ജോലിയുടെ order (സ്വന്തം stack) ഉണ്ട്. വേറൊരു hotel എന്നത് വേറൊരു process ആണ്: അവരുടെ ഫ്രിഡ്ജ് നമുക്ക് നേരിട്ട് തുറക്കാൻ പറ്റില്ല. രണ്ടു cook-മാർ ഒരേ പാത്രത്തിൽ ഒരേ സമയം ഉപ്പിട്ടാൽ കറി കേടാവും; അതാണ് thread-കളിലെ data conflict.

Feature	Process	Thread
Memory	Independent address space	Shared within the process
Communication	Inter-process communication (pipes, sockets, queues)	Direct access to shared data
Overhead	Higher (costly creation and context switching)	Lower (lightweight switching)
Isolation	Strong: a crash or bug in one does not touch another's memory	Weak: needs synchronization
Best for (in Python)	CPU-bound work (multiprocessing)	I/O-bound work (network, files, databases)

Exam tip: "Difference between process and thread" is a favourite 4-5 mark question. Write the definitions first, then at least four rows of this table (memory, communication, overhead, isolation).

Benefits and challenges of threads

Race condition: an error where the result depends on the unpredictable timing of threads that access shared data without synchronization.

Deadlock: a state where two or more threads wait forever for resources held by each other, so none can proceed. **Starvation:** a state where a thread never gets the CPU or a lock because others are always given it first.

Race condition: synchronization ഇല്ലാതെ shared data ഉപയോഗിക്കുന്ന thread-കളുടെ പ്രവചിക്കാനാവാത്ത timing അനുസരിച്ച് result മാറുന്ന തെറ്റ്.

Deadlock: രണ്ടോ അതിലധികമോ thread-കൾ പരസ്പരം പിടിച്ചുവെച്ചിരിക്കുന്ന resources-നായി എന്നെന്നേക്കും കാത്തിരുന്ന് ഒന്നിനും മുന്നോട്ട് പോകാൻ പറ്റാത്ത അവസ്ഥ.

Starvation: മറ്റു thread-കൾക്ക് എപ്പോഴും ആദ്യം കിട്ടുന്നതുകൊണ്ട് ഒരു thread-ന് CPU-വോ lock-ഓ ഒരിക്കലും കിട്ടാത്ത അവസ്ഥ.

Threads give three main benefits. **Responsiveness:** background work runs without freezing the user interface. **Concurrency for I/O:** while one thread waits for a file or network reply, another can work. **Better resource utilization:** the CPU is not idle while tasks wait for external resources. The costs are race conditions, deadlocks, starvation, and, in standard Python, the GIL, which prevents true parallelism for CPU-bound code.

മലയാളത്തിൽ

WhatsApp-ൽ ഒരു വലിയ video download ആക്സോയും നമുക്ക് chat ചെയ്യാൻ പറ്റുന്നത് download വേറൊരു thread-ൽ നടക്കുന്നതുകൊണ്ടാണ്; അതാണ് responsiveness. പക്ഷേ ഗുണങ്ങൾക്കൊപ്പം പ്രശ്നങ്ങളും ഉണ്ട്: shared data തെറ്റിപ്പോകുന്ന race condition, രണ്ടുപേരും പരസ്പരം വഴി മാരാൻ കാത്തുനിൽക്കുന്ന ഇടുങ്ങിയ പാലത്തിലെ രണ്ട് വണ്ടികൾ പോലെയുള്ള deadlock, ക്യൂവിൽ എപ്പോഴും പിന്നിലാവുന്ന ആളെപ്പോലെയുള്ള starvation.

Benefits: responsiveness, concurrency for I/O-bound tasks, better resource utilization

Challenges: race conditions, deadlocks, starvation, GIL (no true parallelism for CPU-bound threads)

Exam tip: The unit describes starvation as a thread that "restarts many times and never gets CPU access". The standard meaning has nothing to do with restarting: starvation is simply a thread that keeps waiting because other threads are always scheduled or given the lock first. Use the definition above.

Thread lifecycle

Thread lifecycle: the sequence of states a thread passes through from creation to termination: New, Runnable, Running, Blocked/Waiting and Terminated.

Thread lifecycle: ഒരു thread ഉണ്ടാക്കിയതു മുതൽ അവസാനിക്കുന്നതു വരെ കടന്നുപോകുന്ന states-ന്റെ ക്രമം: New, Runnable, Running, Blocked/Waiting, Terminated.

New: the Thread object exists but `start()` has not been called. **Runnable:** after `start()` it is ready and waiting for CPU time. **Running:** the scheduler gives it the CPU and it executes its `run()` method (the target function). **Blocked/Waiting:** it is suspended by `sleep()`, I/O, a lock, `wait()` or `join()`, and goes back to Runnable when the wait ends. **Terminated:** `run()` returns or raises an exception; a terminated thread can never be started again.

മലയാളത്തിൽ

ഒരു bus യാത്രക്കാരനെപ്പോലെ ചിന്തിക്കുക. ticket എടുത്തു (New). stop-ൽ bus കാത്തുനിൽക്കുന്നു (Runnable). bus-ൽ കയറി യാത്ര ചെയ്യുന്നു (Running). traffic block-ൽ കുടുങ്ങി (Blocked/Waiting), block മാറിയാൽ വീണ്ടും യാത്ര. സ്ഥലത്തെത്തി ഇറങ്ങി (Terminated). അതേ ticket വെച്ച് വീണ്ടും യാത്ര പറ്റില്ല; അതുപോലെ ഒരു thread-നെ രണ്ടാമത് `start()` ചെയ്യാൻ പറ്റില്ല.



EXAMPLE 1

Trace the states of a thread (from the unit's answer key).

```

from threading import Thread
import time

def worker():
    print("Thread started")      # Running
    time.sleep(1)                # Running -> Blocked/Waiting
    print("Thread resuming")     # back to Running
                                # function returns -> Terminated

t = Thread(target=worker)       # New
print("Thread created (New)")
t.start()                      # New -> Runnable -> Running
t.join()                       # main waits until t is Terminated
print("Thread has terminated")
  
```

```

Thread created (New)
Thread started
Thread resuming
Thread has terminated
  
```

`Thread(...)` എന്ന വരിയിൽ thread New ആണ്. `start()` കഴിഞ്ഞാൽ അത് Runnable, പിന്നെ Running. `sleep(1)` സമയത്ത് Blocked. function തീർന്നാൽ Terminated. `join()` കാരണം അവസാന വരി ഏറ്റവും അവസാനം വരുന്നു.

EXAMPLE 2

What happens if you call `join()` before `start()`, or `start()` twice?

```
from threading import Thread
t = Thread(target=print, args=("hi",))
try:
    t.join()
except RuntimeError as e:
    print("RuntimeError:", e)
t.start(); t.join()
try:
    t.start()
except RuntimeError as e:
    print("RuntimeError:", e)
```

```
RuntimeError: cannot join thread before it is started
hi
RuntimeError: threads can only be started once
```

start ചെയ്യാത്ത thread-നെ join ചെയ്താൽ Python RuntimeError തരും. Terminated ആയ thread-നെ വീണ്ടും start ചെയ്താലും RuntimeError.

Exam tip: The unit's answer key (Check Your Progress 1, Q4) says `join()` before `start()` "returns immediately" and "does not raise an exception". That is wrong: it raises `RuntimeError: cannot join thread before it is started`, as the run above shows. Also, Figure 1 of the unit labels the New → Running arrow `run()`; you must call `start()`. Calling `run()` directly just runs the function in the current thread (see the next topic).

The Global Interpreter Lock (GIL) and thread use cases

GIL (Global Interpreter Lock): a mutex in the CPython interpreter that allows only one thread at a time to execute Python bytecode, even on a multicore processor.

Concurrency: making progress on several tasks by switching between them. **Parallelism:** executing several tasks at literally the same time on different cores.

GIL (Global Interpreter Lock): multicore processor ആണെങ്കിൽ പോലും ഒരു സമയം ഒരൊറ്റ thread-നെ മാത്രം Python bytecode execute ചെയ്യാൻ അനുവദിക്കുന്ന, CPython interpreter-ലെ ഒരു mutex.

Concurrency: ജോലികൾക്കിടയിൽ മാറിമാറി പല ജോലികളിലും പുരോഗതി ഉണ്ടാക്കുന്നത്.

Parallelism: വ്യത്യസ്ത cores-ൽ പല ജോലികളും കൃത്യം ഒരേ സമയത്ത് execute ചെയ്യുന്നത്.

With **sequential** execution tasks run one after another (slowest). With **concurrency** the processor interleaves tasks, using the idle time of one task (for example, waiting for a download) to run another. With **parallelism** tasks truly run at the same time on different cores (fastest). Because of the GIL, Python threads give concurrency, not parallelism, for Python code. The GIL is released while a thread waits for I/O, so threads are still very effective for **I/O-bound** work. For **CPU-bound** work (heavy calculation), use the multiprocessing module: each process has its own interpreter and its own GIL, so processes can use all cores.

മലയാളത്തിൽ

ഒരു ഓഫീസിൽ ഒരൊറ്റ പേന (GIL) മാത്രമേ ഉള്ളൂ എന്ന് കരുതുക. മൂന്ന് clerk-മാർ (threads) ഉണ്ടെങ്കിലും ഒരു സമയം പേന കൈയിലുള്ള ആൾക്കേ എഴുതാൻ പറ്റൂ. ഒരാൾ ഫോണിൽ മറുപടി കാത്തിരിക്കുമ്പോൾ (I/O) പേന മറ്റൊരാൾക്ക് കൊടുക്കും, അതുകൊണ്ട് I/O ജോലികൾക്ക് thread-കൾ ഉപകാരപ്പെടും. പക്ഷേ എല്ലാവർക്കും കണക്കുകൂട്ടൽ (CPU-bound) മാത്രമാണ് ജോലിയെങ്കിൽ ഒരു പേന കൊണ്ട് വേഗം കൂടില്ല; അപ്പോൾ ഓരോരുത്തർക്കും സ്വന്തം ഓഫീസും പേനയും (multiprocessing) കൊടുക്കണം.

I/O-bound task (network, disk, database) → threading

CPU-bound task (heavy computation) → multiprocessing

Use cases of threads: GUI apps, network servers, web scraping, logging, simulations/games

EXAMPLE 1

A CPU-bound countdown, run twice sequentially and then in two threads. The times are from one sample run; your numbers will differ, but they will be about the same as each other.

```
from threading import Thread
import time

def count(n):
    while n > 0:
        n -= 1

N = 20_000_000
start = time.perf_counter()
count(N); count(N)
print(f"Sequential: {time.perf_counter() - start:.2f} s")

start = time.perf_counter()
t1 = Thread(target=count, args=(N,)); t2 = Thread(target=count, args=(N,))
t1.start(); t2.start(); t1.join(); t2.join()
print(f"2 threads : {time.perf_counter() - start:.2f} s")
```

```
Sequential: 2.73 s
2 threads : 2.68 s
```

രണ്ടു thread ഉപയോഗിച്ചിട്ടും സമയം ഏകദേശം അതുതന്നെ. കാരണം GIL: ഒരു സമയം ഒരു thread മാത്രമേ Python code run ചെയ്യൂ. CPU-bound ജോലിക്ക് threading വേഗം കൂട്ടില്ല എന്നതിന്റെ തെളിവാണ്.

EXAMPLE 2

The multiprocessing alternative for CPU-bound work.

```
from multiprocessing import Pool

def square(n):
    return n * n

if __name__ == "__main__":
    with Pool(processes=4) as pool:
        print(pool.map(square, range(8)))
```

```
[0, 1, 4, 9, 16, 25, 36, 49]
```

ഇവിടെ 4 process-കൾ ഉണ്ടാകുന്നു, ഓരോന്നിനും സ്വന്തം GIL. `pool.map` result-കൾ input-ന്റെ അതേ ക്രമത്തിൽ തിരിച്ചുതരും. Windows-ലും macOS-ലും `if __name__ == "__main__":` guard നിർബന്ധമാണ്.

Exam tip: The GIL belongs to CPython (the standard interpreter), not to the Python language. Since Python 3.13 an optional "free-threaded" CPython build without the GIL exists, but the normal build still has it, so the unit's statements remain the right exam answer.

Creating and starting a thread

threading module: Python's standard-library module for creating and managing threads; a thread is an object of its `Thread` class.

start(): the method that begins a new thread of execution, which then calls the thread's `run()` method (the target function).

threading module: thread-കൾ ഉണ്ടാക്കാനും കൈകാര്യം ചെയ്യാനുമുള്ള Python standard-library module; ഒരു thread അതിലെ `Thread` class-ന്റെ object ആണ്.

start(): execution-ന്റെ ഒരു പുതിയ thread തുടങ്ങുന്ന method; ആ thread പിന്നെ സ്വന്തം `run()` method (target function) call ചെയ്യുന്നു.

There are two ways. (1) Pass a function: `Thread(target=func, args=(...), kwargs={...})`, then call `start()`. (2) Subclass `Thread` and override `run()` (the object-oriented way). Always call `start()`, never `run()` directly. After `start()`, the main thread continues at once; the operating system decides when each thread runs, so without coordination the print order changes from run to run.

മലയാളത്തിൽ

ഒരു ജോലിക്കാരനെ നിയമിക്കുന്നത് (`Thread(target=...)`) പോലെയാണ് `thread` object ഉണ്ടാക്കുന്നത്; "തുടങ്ങിക്കോ" എന്ന് പറയുന്നതാണ് `start().run()` നേരിട്ട് വിളിച്ചാൽ ആ ജോലി നമ്മൾ തന്നെ ചെയ്യുന്നതുപോലെയാണ്, പുതിയ `thread` ഉണ്ടാവുന്നില്ല. ആദ്യം `print` ചെയ്യും എന്ന് OS scheduler ആണ് തീരുമാനിക്കുന്നത്, അതുകൊണ്ട് output ക്രമം ഓരോ run-ലും മാറാം.

```
from threading import Thread, current_thread
t = Thread(target=function_name, args=(a, b), kwargs={'k': v}, name="...", daemon=False)
t.start() # new thread runs function_name(a, b, k=v)
```

EXAMPLE 1

The unit's program: a worker thread and the main thread both print. This is one sample run; the order may differ on your machine and between runs.

```
from threading import Thread, current_thread

def display(n, msg):
    for i in range(n):
        print("Thread:", current_thread().name, "->", msg)

t1 = Thread(target=display, kwargs={'n': 3, 'msg': "Hello World!"})
t1.start()
for i in range(3):
    # main thread continues at once
    print("Thread:", current_thread().name, "-> welcome")
```

```
Thread: Thread-1 (display) -> Hello World!
Thread: Thread-1 (display) -> Hello World!
Thread: Thread-1 (display) -> Hello World!
Thread: MainThread -> welcome
Thread: MainThread -> welcome
Thread: MainThread -> welcome
```

ഈ run-ൽ worker thread ആദ്യം തീർന്നു; മറ്റൊരു run-ൽ വരികൾ ഇടകലർന്ന് വരാം. worker thread-ന്റെ default പേര് `Thread-1 (display)` ആണ് (Python 3.10 മുതൽ target function-ന്റെ പേരും ചേർക്കും).

EXAMPLE 2

Make the output deterministic: wait for the worker with `join()`.

```
from threading import Thread, current_thread

def display(n, msg):
    for i in range(n):
        print("Thread:", current_thread().name, "->", msg)

t1 = Thread(target=display, kwargs={'n': 2, 'msg': "Hello World!"})
t1.start()
t1.join()
display(2, "welcome")
```

```
Thread: Thread-1 (display) -> Hello World!
Thread: Thread-1 (display) -> Hello World!
Thread: MainThread -> welcome
Thread: MainThread -> welcome
```

EXAMPLE 3

Create a thread by subclassing Thread, and compare `run()` with `start()`.

```
from threading import Thread, current_thread

class Greeter(Thread):
    def __init__(self, person):
        super().__init__()          # must call Thread.__init__
        self.person = person

    def run(self):                   # code the new thread executes
        print("Hello,", self.person, "from", self.name)

g = Greeter("Anu")
g.start()
g.join()

def show():
    print("Running in:", current_thread().name)
t = Thread(target=show)
t.run()                            # wrong way: no new thread
t2 = Thread(target=show)
t2.start(); t2.join()              # right way
```

```
Hello, Anu from Thread-1
Running in: MainThread
Running in: Thread-3 (show)
```

Subclass രീതിയിൽ `run()` override ചെയ്യണം, `__init__`-ൽ `super().__init__()` വിളിക്കാൻ മറക്കരുത്. `t.run()` MainThread-ൽ തന്നെ run ആയി; `start()` വിളിച്ചപ്പോൾ മാത്രമാണ് പുതിയ thread ഉണ്ടായത്.

Exam tip: The unit's code has the typo `current_ thread` (with a space); the correct name is `current_thread`. Its sample output also shows only 3 iterations although `n` is 4. If asked "why is the output order different each time?", answer: the OS thread scheduler decides when threads switch.

Thread names and identifiers

Thread name: a string stored in the `.name` attribute, "MainThread" for the main thread and "Thread-N" by default for others; used for debugging and can be changed.

ident: a non-zero integer the Python interpreter assigns when a thread starts. **native_id:** the thread ID assigned by the operating system.

Thread name: `.name` attribute-ൽ സൂക്ഷിക്കുന്ന string; main thread-ന് "MainThread", മറ്റുള്ളവക്ക് default ആയി "Thread-N"; debugging-ന് ഉപയോഗിക്കുന്നു, മാറ്റാനും പറ്റും.

ident: thread start ആകുമ്പോൾ Python interpreter കൊടുക്കുന്ന പുഷ്പമല്ലാത്ത ഒരു integer.

native_id: operating system കൊടുക്കുന്ന thread ID.

A name can be given with `Thread(..., name="Downloader")` or by assigning `t.name = "Worker-1"`; even the main thread can be renamed. `ident` is read-only, is `None` before the thread starts, and may be reused by a new thread after the old one ends. `native_id` is the number the OS shows in tools like `top`. The process itself has a process ID from `os.getpid()`; all threads of a process share the same PID.

മലയാളത്തിൽ

ഒരു ക്ലാസിലെ കുട്ടികൾക്ക് പേരും (name) roll number-ഉം (ident) ഉള്ളതുപോലെ. പേര് നമ്മുടെ സൗകര്യത്തിന് മാറ്റാം; roll number interpreter കൊടുക്കുന്നതാണ്, മാറ്റാൻ പറ്റില്ല. `native_id` എന്നത് സർക്കാർ (OS) കൊടുക്കുന്ന Aadhaar number പോലെ. സ്കൂളിന്റെ number ആണ് PID, അത് ഒരു process-ലെ എല്ലാ thread-കൾക്കും ഒന്നുതന്നെ.

EXAMPLE

Naming threads and reading their identifiers. The real ID numbers change each run, so the program prints checks instead of the raw numbers.

```
from threading import Thread, current_thread
import threading, os

def work():
    me = current_thread()
    print(me.name, "ident is int:", isinstance(me.ident, int),
          "| native_id matches:", me.native_id == threading.get_native_id())

t1 = Thread(target=work)
print("Default name:", t1.name)
t1.name = "Worker-1"
t2 = Thread(target=work, name="Downloader")
print("Before start, ident =", t1.ident)
t1.start(); t1.join()
t2.start(); t2.join()
current_thread().name = "NewMainThread"
print("Main thread is now:", current_thread().name)
print("PID is int:", isinstance(os.getpid(), int))
```

```
Default name: Thread-1 (work)
Before start, ident = None
Worker-1 ident is int: True | native_id matches: True
Downloader ident is int: True | native_id matches: True
Main thread is now: NewMainThread
PID is int: True
```

start ചെയ്യുന്നതിനു മുമ്പ് ident None ആണ്. `print(t1.ident)` എന്ന് നോക്കിട്ട് എഴുതിയാൽ ഓരോ run-ലും വ്യത്യസ്തമായ ഒരു വലിയ സംഖ്യ കാണാം.

Useful thread functions

is_alive(): a Thread method that returns True from the moment `start()` is called until `run()` finishes.

active_count(): returns the number of Thread objects currently alive. **enumerate():** returns a list of all Thread objects currently alive.

is_alive(): `start()` വിളിച്ച നിമിഷം മുതൽ `run()` തീരുന്നതു വരെ True തിരിച്ചുതന്ന Thread method.

active_count(): ഇപ്പോൾ alive ആയ Thread objects-ന്റെ എണ്ണം തരുന്നു. **enumerate():** ഇപ്പോൾ alive ആയ എല്ലാ Thread objects-ന്റെയും list തരുന്നു.

`is_alive()` is called on a thread object (`t.is_alive()`). The others are module functions:

`threading.main_thread()` returns the main thread object, `threading.current_thread()` the calling thread, `threading.active_count()` the number of alive threads (including the main thread), `threading.enumerate()` the list of alive threads, and `threading.get_native_id()` the OS ID of the calling thread.

മലയാളത്തിൽ

ഇവ ഒരു manager-ന്റെ attendance register പോലെയാണ്: ആരൊക്കെ ഇപ്പോൾ ജോലിയിലുണ്ട് (`enumerate()`), എത്ര പേർ (`active_count()`), ഒരാൾ ഇപ്പോഴും ജോലി ചെയ്യുന്നുണ്ടോ (`is_alive()`), മുതലാളി ആര് (`main_thread()`).

EXAMPLE

Watch a thread before, during and after it runs.

```
import threading, time

def sleeper():
    time.sleep(0.5)

t = threading.Thread(target=sleeper, name="Sleeper")
print("Alive before start:", t.is_alive())
t.start()
print("Alive after start:", t.is_alive())
print("Active threads:", threading.active_count())
print("Names:", sorted(th.name for th in threading.enumerate()))
print("Main thread:", threading.main_thread().name)
t.join()
print("Alive after join:", t.is_alive())
print("Active threads:", threading.active_count())
```

```
Alive before start: False
Alive after start: True
Active threads: 2
Names: ['MainThread', 'Sleeper']
Main thread: MainThread
Alive after join: False
Active threads: 1
```

Sleeper 0.5 സെക്കൻഡ് ഉറങ്ങുന്നതുകൊണ്ട് start കഴിഞ്ഞ ഉടനെ അത് alive ആണ്, എണ്ണം 2 (MainThread + Sleeper). join കഴിഞ്ഞാൽ അത് Terminated, എണ്ണം 1.

The join() method

join(): a Thread method that blocks the calling thread until the thread it is called on terminates (or until an optional timeout expires).

join(): ഏത് thread-ന്മേലാണോ വിളിക്കുന്നത് അത് terminate ആകുന്നതു വരെ (അല്ലെങ്കിൽ optional timeout തീരുന്നതു വരെ) വിളിക്കുന്ന thread-നെ block ചെയ്യുന്ന Thread method.

`t.join()` means "wait here until `t` finishes". Use it when one step depends on another's result, or so that the main thread prints the final result only after all workers are done. The usual pattern is: start all threads first, then join all of them, so they still run concurrently. `t.join(timeout=s)` waits at most `s` seconds; it always returns `None`, so check `t.is_alive()` afterwards to see whether the thread really finished.

മലയാളത്തിൽ

വീട്ടിൽ എല്ലാവരും ഒരുമിച്ച് ഉണ് കഴിക്കാൻ ഇരിക്കുമ്പോൾ അച്ഛൻ വരുന്നതു വരെ കാത്തിരിക്കുന്നതുപോലെയാണ് `join()`. എല്ലാ thread-കളും ആദ്യം `start` ചെയ്ത്, പിന്നീട് എല്ലാറ്റിനെയും `join` ചെയ്താൽ ജോലികൾ ഒരേ സമയം നടക്കും, `main thread` അവസാനം വരെ കാത്തിരിക്കുകയും ചെയ്യും.

EXAMPLE 1

Force task 1 to finish before task 2 starts, and the main thread to finish last.

```
from threading import Thread
import time

def task1():
    print("Task 1 started")
    time.sleep(1)
    print("Task 1 finished")

def task2():
    print("Task 2 started")
    time.sleep(0.5)
    print("Task 2 finished")

t1 = Thread(target=task1)
t2 = Thread(target=task2)
t1.start()
t1.join()          # wait for task1 before starting task2
t2.start()
t2.join()          # wait for task2 before main continues
print("Main thread ends last")
```

```
Task 1 started
Task 1 finished
Task 2 started
Task 2 finished
Main thread ends last
```

EXAMPLE 2

Start-all-then-join-all keeps the threads concurrent; `join(timeout)` may return while the thread is still alive.

```
from threading import Thread
import time

def work(sec):
    time.sleep(sec)

start = time.perf_counter()
threads = [Thread(target=work, args=(1,)) for _ in range(3)]
for t in threads:
    t.start()
for t in threads:
    t.join()
print(f"3 tasks of 1 s each took about {time.perf_counter() - start:.0f} s")

t = Thread(target=work, args=(3,))
t.start()
t.join(timeout=0.5)
print("Still alive after join(timeout=0.5)?", t.is_alive())
t.join()
```

```
3 tasks of 1 s each took about 1 s
Still alive after join(timeout=0.5)? True
```

മൂന്നു ജോലിയും ഒരേ സമയം ഉറങ്ങിയതുകൊണ്ട് 3 സെക്കൻഡിനു പകരം 1 സെക്കൻഡ് മതിയായി. `start()`-ഉം `join()`-ഉം ഒരേ loop-ൽ എഴുതിയാൽ ജോലികൾ ഒന്നിനു പിറകെ ഒന്നായി മാറും (3 സെക്കൻഡ്).

Exam tip: A common mistake is for `t in threads: t.start(); t.join()` in one loop: this runs the threads one at a time and removes all concurrency.

Daemon and non-daemon threads

Daemon thread: a background thread that is stopped automatically when all non-daemon threads have finished, so it never prevents the program from exiting.

Non-daemon (user) thread: a thread doing essential work; the program waits for it to finish before exiting.

Daemon thread: എല്ലാ non-daemon thread-കളും തീരുമ്പോൾ തനിയെ നിർത്തപ്പെടുന്ന, അതുകൊണ്ട് program exit ആകുന്നത് ഒരിക്കലും തടയാത്ത, background thread.

Non-daemon (user) thread: അത്യാവശ്യ ജോലി ചെയ്യുന്ന thread; exit ആകുന്നതിനു മുമ്പ് program അത് തീരാൻ കാത്തിരിക്കും.

Make a thread a daemon with `Thread(..., daemon=True)` or `t.daemon = True` before `start()`. Daemon threads suit background services such as log writers, monitors and cache cleaners, typically running an endless `while True` loop. Critical work (saving a file, completing a payment) must be done in a non-daemon thread: a daemon is killed abruptly at exit, possibly in the middle of its work, and its finally blocks may not run.

മലയാളത്തിൽ

കല്യാണ ഹാളിലെ പശ്ചാത്തല സംഗീതം ഒരു daemon thread പോലെയാണ്: പരിപാടി കഴിഞ്ഞ് എല്ലാവരും പോകുമ്പോൾ അത് തനിയെ നിർത്തും; പാട്ട് തീരാൻ ആരും കാത്തുനിൽക്കില്ല. പക്ഷേ താലികെട്ട് (critical work) തീരാതെ ആരും പോകില്ല; അതാണ് non-daemon thread. പണം കൈമാറ്റം പോലുള്ള ജോലി daemon ആക്കിയാൽ പകുതിയിൽ മുറിഞ്ഞുപോകാം.

Daemon thread	Non-daemon (user) thread
Background, supporting work	Main, essential work
Program does not wait for it	Program waits for it to finish
Killed abruptly at exit	Always runs to completion
Logging, monitoring, auto-save, heartbeat	Saving data, transactions, computations
daemon=True	Default (daemon=False, inherited from the creating thread)

EXAMPLE

A daemon logger runs forever; a non-daemon worker saves a file. The program ends when the worker and the main thread are done.

```
from threading import Thread
import time

def logger():
    while True:
        print("[Logger] writing logs...")
        time.sleep(1)

def worker():
    print("Worker: saving file")
    time.sleep(2.5)
    print("Worker: file saved")

lg = Thread(target=logger, name="Logger", daemon=True)
w = Thread(target=worker)
print("Logger daemon?", lg.daemon, "| Worker daemon?", w.daemon)
lg.start()
time.sleep(0.1)
w.start()
w.join()
print("Main: exiting, the daemon logger is stopped automatically")
```

```
Logger daemon? True | Worker daemon? False
[Logger] writing logs...
Worker: saving file
[Logger] writing logs...
[Logger] writing logs...
Worker: file saved
Main: exiting, the daemon logger is stopped automatically
```

Logger 0, 1, 2 സെക്കൻഡുകളിൽ എഴുതുന്നു; worker 2.6-ൽ തീരുന്നു. main thread-ഉം worker-ഉം തീർന്നപ്പോൾ അനന്തമായ logger loop program-നെ പിടിച്ചുനിർത്തിയില്ല. daemon=True ഇല്ലായിരുന്നെങ്കിൽ ഈ program ഒരിക്കലും അവസാനിക്കില്ല.

Exam tip: "Why not make a critical computation a daemon?" Answer: the program may exit before the daemon finishes, so its results (for example, a half-written file) are lost. "Why should a log writer be a daemon?" Answer: logging is secondary and runs in an endless loop; as a non-daemon it would stop the program from ever exiting.

Thread synchronization and race conditions

Thread synchronization: coordinating threads so that shared data is accessed in a safe, controlled order.

Critical section: a part of code that accesses shared data and must be executed by only one thread at a time.

Thread synchronization: shared data സുരക്ഷിതവും നിയന്ത്രിതവുമായ ക്രമത്തിൽ ഉപയോഗിക്കപ്പെടാൻ thread-കളെ ഏകോപിപ്പിക്കുന്നത്.

Critical section: shared data ഉപയോഗിക്കുന്നതും ഒരു സമയം ഒരു thread മാത്രം execute ചെയ്യേണ്ടതുമായ code ഭാഗം.

When threads share a global variable, file or buffer without coordination, the result can be wrong. Updating a counter is really three steps: **read** the value, **add 1**, **write** it back. If thread A reads 5, then thread B also reads 5, both write 6: one increment is lost. Python's tools to prevent this are Lock, RLock, Condition and Semaphore (and the thread-safe queue .Queue).

മലയാളത്തിൽ

ഒരു ബാങ്ക് അക്കൗണ്ടിൽ 5000 രൂപ ഉണ്ട്. ഭർത്താവും ഭാര്യയും രണ്ട് ATM-ൽ ഒരേ സമയം balance നോക്കി, രണ്ടുപേരും 5000 കണ്ടു. ഓരോരുത്തരും 1000 നിക്ഷേപിച്ച് "6000" എന്ന് എഴുതുന്നു. ശരിക്കും 7000 വേണ്ടിയിരുന്നു; ഒരു update നഷ്ടപ്പെട്ടു. ഇതാണ് race condition: ആരുടെ timing ആദ്യം എന്നതനുസരിച്ച് ഉത്തരം മാറുന്നു.

`counter += 1` $\equiv$ `temp = counter` (read) $\rightarrow$ `temp = temp + 1` (modify) $\rightarrow$ `counter = temp` (write)

Race: A reads 5, B reads 5, A writes 6, B writes 6 $\rightarrow$ one increment lost

EXAMPLE

The unit's race-condition demo: two threads each add 1 to a shared counter 1000 times. `time.sleep(0)` gives up the CPU between read and write, which makes a thread switch at the worst moment very likely. Three runs gave these results (yours will differ, but will almost always be below 2000):

```
from threading import Thread
import time

counter = 0                # shared resource

def increment(n):
    global counter
    for _ in range(n):
        temp = counter      # read
        temp += 1           # modify
        time.sleep(0)       # yield: invites a thread switch here
        counter = temp      # write back

t1 = Thread(target=increment, args=(1000,))
t2 = Thread(target=increment, args=(1000,))
t1.start(); t2.start()
t1.join(); t2.join()
print("Final counter value:", counter)
```

```
Final counter value: 1002
Final counter value: 1000
Final counter value: 1002
```

2000 കിട്ടേണ്ടിയിട്ട് ഏകദേശം 1000 മാത്രം. ഒരു thread read ചെയ്ത് ശേഷം ഉറങ്ങുന്നു, ആ സമയം മറ്റേ thread അതേ പഴയ value വായിക്കുന്നു, പിന്നീട് രണ്ടുപേരും ഒരേ value എഴുതുന്നു. അങ്ങനെ പകുതിയോളം increments നഷ്ടപ്പെടുന്നു. അടുത്ത topic-ൽ Lock ഉപയോഗിച്ച് ഇത് ശരിയാക്കാം.

Exam tip: Even though the GIL runs only one thread at a time, it does **not** prevent race conditions: a switch can still happen between the read and the write. Always protect the whole read-modify-write with a lock.

Locks (mutexes) and deadlock

Lock (mutex): a synchronization object that enforces mutual exclusion: only the one thread that has acquired it may enter the critical section, until it releases it.

Mutual exclusion: the guarantee that at most one thread executes a critical section at any time.

Lock (mutex): mutual exclusion ഉറപ്പാക്കുന്ന synchronization object: അത് acquire ചെയ്ത ഒരൊറ്റ thread-ന് മാത്രമേ, അത് release ചെയ്യുന്നതു വരെ, critical section-ൽ കയറാൻ പറ്റൂ.

Mutual exclusion: ഏതു സമയത്തും പരമാവധി ഒരു thread മാത്രമേ ഒരു critical section execute ചെയ്യൂ എന്ന ഉറപ്പ്.

A Lock has two states, locked and unlocked. `acquire()` takes it (blocking if another thread holds it) and `release()` gives it back. The `with lock:` statement acquires on entry and releases on exit, even if an exception occurs, so it is the preferred form. Keep the critical section as small as possible. `acquire(timeout=s)` returns `False` if the lock could not be taken in `s` seconds.

മലയാളത്തിൽ

ബസ് സ്റ്റാൻഡിലെ ഒരു ടോയ്ലറ്റും അതിന്റെ ഒരൊറ്റ താക്കോലും പോലെയാണ് Lock. താക്കോൽ കിട്ടിയ ആൾ മാത്രം അകത്ത് (acquire); പുറത്തിറങ്ങുമ്പോൾ താക്കോൽ തിരിച്ചുകൊടുക്കും (release); ബാക്കിയുള്ളവർ വരിയിൽ കാത്തുനിൽക്കും. `with lock:` എഴുതിയാൽ താക്കോൽ തിരിച്ചുകൊടുക്കാൻ മറക്കില്ല, error വന്നാലും.

```
lock = Lock()
with lock: # lock.acquire()
    critical section
    # lock.release() automatically, even on error
```

EXAMPLE 1

Fix the race condition of the previous topic with a Lock. The same slow read-sleep-write code is now inside with lock:. Three runs:

```
from threading import Thread, Lock
import time

counter = 0
lock = Lock()

def safe_increment(n):
    global counter
    for _ in range(n):
        with lock:                # only one thread at a time here
            temp = counter
            temp += 1
            time.sleep(0)         # a switch here is now harmless
            counter = temp

t1 = Thread(target=safe_increment, args=(1000,))
t2 = Thread(target=safe_increment, args=(1000,))
t1.start(); t2.start()
t1.join(); t2.join()
print("Final counter value with Lock:", counter)
```

```
Final counter value with Lock: 2000
Final counter value with Lock: 2000
Final counter value with Lock: 2000
```

ഇപ്പോൾ ഒരു thread read-modify-write മുഴുവനായി തീർക്കുന്നത് വരെ മറ്റേ thread-ന് lock കിട്ടില്ല. ഉറങ്ങുന്ന സമയത്ത് switch നടന്നാലും മറ്റേ thread lock-നായി കാത്തുനിൽക്കുകയേ ഉള്ളൂ. അതുകൊണ്ട് എപ്പോഴും 2000.

EXAMPLE 2

Deadlock with two locks, and how a fixed lock order avoids it. When T1 and T2 take the locks in opposite orders, each holds one and waits for the other; a timeout lets T1 give up. When both take them in the same order (a then b), no circular wait is possible.

```
from threading import Thread, Lock
import time

lock_a, lock_b = Lock(), Lock()

def task(name, first, second, wait):
    with first:
        time.sleep(0.1)
        if second.acquire(timeout=wait):
            print(name, "got both locks")
            second.release()
        else:
            print(name, "gave up after", wait, "s (would deadlock)")

print("Opposite order:")
t1 = Thread(target=task, args=("T1", lock_a, lock_b, 1))
t2 = Thread(target=task, args=("T2", lock_b, lock_a, 3))
t1.start(); t2.start(); t1.join(); t2.join()

print("Same order (a then b):")
t1 = Thread(target=task, args=("T1", lock_a, lock_b, 1))
t2 = Thread(target=task, args=("T2", lock_a, lock_b, 1))
t1.start(); time.sleep(0.01); t2.start(); t1.join(); t2.join()
```

```
Opposite order:
T1 gave up after 1 s (would deadlock)
T2 got both locks
Same order (a then b):
T1 got both locks
T2 got both locks
```

ആദ്യ ഭാഗത്ത് T1-ന്റെ കൈയിൽ lock_a, T2-ന്റെ കൈയിൽ lock_b; രണ്ടുപേരും മറ്റേതിനായി കാത്തിരിക്കുന്നു. timeout ഇല്ലായിരുന്നെങ്കിൽ program എന്നെന്നേക്കും നിന്നുപോകും (deadlock). T1 ഉപേക്ഷിച്ച് lock_a വിട്ടപ്പോൾ T2 മുന്നോട്ടുപോയി. രണ്ടാം ഭാഗത്ത് എല്ലാവരും ഒരേ ക്രമത്തിൽ lock എടുക്കുന്നതുകൊണ്ട് deadlock സാധ്യമല്ല.

Avoiding deadlock with several locks:

1. Fixed order: every thread acquires locks in the same global order (lock1 → lock2 → lock3)
2. Timeouts: `acquire(timeout=s)`; on failure release what you hold and retry
3. Fewer locks: hold one lock at a time, or use a higher-level tool such as `queue.Queue`

Exam tip: The unit's answer key (Check Your Progress 2, Q5) suggests "use Queue or RLock instead of multiple locks". Queue is right; RLock is not a cure for deadlock between different locks. It only stops a thread from blocking on a lock it already holds. Two threads taking two RLocks in opposite orders still deadlock.

Reentrant locks (RLock)

RLock (reentrant lock): a lock that the thread already holding it can acquire again without blocking; it is freed only after that thread releases it as many times as it acquired it.

RLock (reentrant lock): അത് ഇപ്പോൾ കൈവശമുള്ള thread-ന് block ആകാതെ വീണ്ടും acquire ചെയ്യാവുന്ന lock; എത്ര തവണ acquire ചെയ്തോ അത്ര തവണ ആ thread release ചെയ്താൽ മാത്രമേ അത് സ്വതന്ത്രമാകൂ.

A normal Lock does not remember who holds it, so if the same thread calls `acquire()` again it blocks forever, waiting for itself. An RLock stores the **owner thread** and a **recursion count**: the owner may re-acquire (count goes up), each release lowers the count, and other threads can take it only when the count is 0. This is needed when a function holding a lock calls another function (or itself, recursively) that takes the same lock. Only the owner may release an RLock.

മലയാളത്തിൽ

വീടിന്റെ പ്രധാന വാതിൽ തുറന്ന് അകത്ത് കയറിയ ആൾ അകത്തെ മുറിയിലേക്ക് പോകുമ്പോൾ "താക്കോൽ ഇവിടെ ഇല്ലല്ലോ" എന്ന് പറഞ്ഞ് പുറത്ത് കാത്തുനിൽക്കുന്നതു പോലെയാണ് സാധാരണ Lock-ലെ പ്രശ്നം: സ്വന്തം കൈയിലുള്ള താക്കോലിനായി സ്വയം കാത്തിരിക്കുന്നു. RLock "ഇത് നീ തന്നെയല്ലേ, കയറിക്കോ" എന്ന് തിരിച്ചറിയും, എത്ര വാതിൽ കയറി എന്ന് എണ്ണിവെക്കുകയും ചെയ്യും.

Lock	RLock
Same thread acquiring twice → blocks forever (self-deadlock)	Same thread may acquire many times
No owner; any thread may release	Has an owner; only the owner may release
No count	Keeps a recursion count; free when count = 0
Slightly faster, simpler	For nested or recursive locking

EXAMPLE 1

A plain Lock acquired twice by the same thread (a timeout is used so the demo does not hang).

```
from threading import Lock

lock = Lock()
lock.acquire()
print("First acquire done")
got = lock.acquire(timeout=1)  # same thread tries again
print("Second acquire succeeded?", got)
lock.release()
```

```
First acquire done
Second acquire succeeded? False
```

EXAMPLE 2

The unit's nested-function example with an RLock.

```
from threading import Thread, RLock

lock = RLock()

def outer_task():
    with lock:
        print("Outer acquired lock, calling inner...")
        inner_task()

def inner_task():
    with lock:          # same thread re-enters safely
        print("Reentrant lock acquired safely")

t = Thread(target=outer_task)
t.start()
t.join()
```

```
Outer acquired lock, calling inner...
Reentrant lock acquired safely
```

`RLock()`—ന പകരം `Lock()` ആയിരുന്നെങ്കിൽ `inner_task`-ലെ `with lock:` എന്നെന്തേക്കും `block` ആകുമായിരുന്നു, `program` നിന്നുപോകും.

EXAMPLE 3

An OOP case: a class whose locked method calls another locked method.

```
from threading import RLock

class Account:
    def __init__(self, balance):
        self.balance = balance
        self.lock = RLock()

    def withdraw(self, amt):
        with self.lock:
            self.balance -= amt

    def transfer_fee(self, amt, fee):
        with self.lock:          # holds the lock ...
            self.withdraw(amt)    # ... and withdraw() takes it again
            self.withdraw(fee)

acc = Account(1000)
acc.transfer_fee(200, 10)
print("Balance:", acc.balance)
```

Balance: 790

1000 – 200 – 10 = 790. transfer_fee lock പിടിച്ചുകൊണ്ട് withdraw വിളിക്കുന്നു, withdraw വീണ്ടും അതേ lock എടുക്കുന്നു. RLock ആയതുകൊണ്ട് പ്രശ്നമില്ല.

Condition variables

Condition variable: a synchronization object, combining a lock with wait/notify, that lets threads sleep until another thread signals that a particular state has changed.

Condition variable: ഒരു lock-ഉം wait/notify-ഉം ചേർന്ന synchronization object; ഒരു പ്രത്യേക state മാറി എന്ന് മറ്റൊരു thread signal ചെയ്യുന്നതു വരെ thread-കളെ ഉറങ്ങാൻ അനുവദിക്കുന്നു.

All methods are used while holding the condition's lock (with condition:). wait() releases the lock and sleeps until notified, then re-acquires the lock before returning. notify() wakes one waiting thread; notify_all() wakes all of them. Always check the condition in a while loop, not an if, because a woken thread must re-check that the state really is what it needs. This avoids **busy waiting** (looping and checking again and again), so waiting threads use no CPU.

മലയാളത്തിൽ

ചായക്കൂടയിൽ ചായ റെഡിയാകുമ്പോൾ കടക്കാരുടെ "ചായ റെഡി!" എന്ന് വിളിച്ചുപറയും വരെ നമ്മൾ ബെഞ്ചിൽ ഇരിക്കുന്നു (`wait()`); ഓരോ സെക്കൻഡിലും എഴുന്നേറ്റ് ചെന്ന് "ആയോ?" എന്ന് ചോദിക്കുന്നില്ല (`busy waiting` ഇല്ല). വിളി കേട്ടാൽ (`notify()`) എഴുന്നേറ്റ് ചെന്ന് ഗ്ലാസ് ശരിക്കും ഉണ്ടോ എന്ന് ഒന്നുകൂടി നോക്കുന്നു; അതാണ് `while` loop.

Consumer: with cond: while not ready: cond.wait() → use data

Producer: with cond: change data → cond.notify() (or `notify_all()`)

EXAMPLE

The unit's producer-consumer with a Condition. A short delay in the producer makes the consumer wait first, so the output is always the same.

```
from threading import Thread, Condition
import time

condition = Condition()
buffer = []

def producer():
    time.sleep(0.5)          # consumer starts waiting first
    with condition:
        buffer.append("item")
        print("Producer added item")
        condition.notify()   # wake one waiting consumer

def consumer():
    with condition:
        while not buffer:    # re-check after every wake-up
            print("Consumer waiting...")
            condition.wait()  # releases lock, sleeps
        item = buffer.pop()
        print("Consumer got:", item)

c = Thread(target=consumer)
p = Thread(target=producer)
c.start(); p.start()
c.join(); p.join()
```

```
Consumer waiting...
Producer added item
Consumer got: item
```

Consumer buffer കാലിയാൽ കണ്ട് `wait()` വിളിക്കുന്നു; ഇത് lock വിട്ടുകൊടുക്കുന്നതുകൊണ്ട് producer-ന് lock എടുത്ത് item ചേർക്കാൻ പറ്റുന്നു. `notify()` കേട്ട് consumer ഉണർന്ന്, lock വീണ്ടും എടുത്ത്, buffer നോക്കി, item എടുക്കുന്നു.

Exam tip: For a bounded buffer, the producer also waits (`while len(buffer) == MAX: cond.wait()`) and the consumer notifies after removing an item. Mention that `wait()` releases the lock; that is the key point examiners look for.

Semaphores

Semaphore: a synchronization object with an internal counter that allows at most n threads to access a resource at the same time; `acquire()` decrements the counter and blocks at 0, `release()` increments it.

Semaphore: ഒരു internal counter ഉള്ള synchronization object; ഒരു resource ഒരേ സമയം പരമാവധി n thread-കൾക്ക് മാത്രം ഉപയോഗിക്കാൻ അനുവദിക്കുന്നു; `acquire()` counter കുറയ്ക്കുകയും 0-ൽ block ചെയ്യുകയും, `release()` counter കൂട്ടുകയും ചെയ്യുന്നു.

`Semaphore(n)` starts its counter at n . A thread entering with `sem`: takes one permit; when the counter is 0, further threads wait until a permit is released. It is used to limit concurrent access to a limited resource such as database connections, downloads or bandwidth. A Lock allows exactly one thread; a Semaphore allows up to n . `BoundedSemaphore` additionally raises an error if `release()` is called more times than `acquire()`.

മലയാളത്തിൽ

ATM counter-ൽ 2 മെഷീൻ ഉണ്ടെന്ന് കരുതുക. ഒരേ സമയം 2 പേർക്ക് മാത്രം അകത്ത് കയറാം; ബാക്കി 3 പേർ പുറത്ത് കാത്തുനിൽക്കും. ഒരാൾ ഇറങ്ങുമ്പോൾ അടുത്തയാൾ കയറും. Lock എന്നത് ഒരു മെഷീൻ മാത്രമുള്ള ATM ആണ് (`Semaphore(1)`-നു സമാനം).

Lock	Semaphore
One thread at a time	Up to n threads at a time
Like a single key	Like n keys (permits)
Protects shared data (mutual exclusion)	Limits use of a resource pool

EXAMPLE 1

The unit's example with 5 threads and Semaphore (2) (sleep shortened to 1 s). This is one sample run; which thread enters next may differ between runs, but never more than two are inside at once.

```
from threading import Thread, Semaphore
import time

sem = Semaphore(2)

def task(i):
    with sem:
        print("Thread", i, "entered")
        time.sleep(1)
        print("Thread", i, "leaving")

for i in range(5):
    Thread(target=task, args=(i,)).start()
```

```
Thread 0 entered
Thread 1 entered
Thread 0 leaving
Thread 2 entered
Thread 1 leaving
Thread 3 entered
Thread 2 leaving
Thread 4 entered
Thread 3 leaving
Thread 4 leaving
```

EXAMPLE 2

Prove the limit with a deterministic output: count how many threads are inside at the same time.

```
from threading import Thread, Semaphore, Lock
import time

sem = Semaphore(2)
inside = 0
peak = 0
count_lock = Lock()

def task(i):
    global inside, peak
    with sem:
        with count_lock:
            inside += 1
            peak = max(peak, inside)
        time.sleep(0.5)
        with count_lock:
            inside -= 1

start = time.perf_counter()
threads = [Thread(target=task, args=(i,)) for i in range(5)]
for t in threads:
    t.start()
for t in threads:
    t.join()
print("Most threads inside at once:", peak)
print(f"Time for 5 jobs of 0.5 s: about {time.perf_counter() - start:.1f} s")
```

```
Most threads inside at once: 2
Time for 5 jobs of 0.5 s: about 1.5 s
```

5 ജോലികൾ 2-2-1 എന്ന batches ആയി നടക്കുന്നു: $3 \text{ round} \times 0.5 = 1.5$ സെക്കൻഡ്. inside എന്ന shared counter-ഉം ഒരു Lock കൊണ്ട് സംരക്ഷിച്ചിട്ടുണ്ട് എന്ന് ശ്രദ്ധിക്കുക; semaphore 2 thread-നെ കയറ്റുന്നതുകൊണ്ട് അതിനുള്ളിൽ race condition ഇപ്പോഴും സാധ്യമാണ്.

Thread pools (concurrent.futures)

Thread pool: a fixed group of reusable worker threads to which many tasks are submitted; each free worker takes the next queued task.

concurrent.futures: a module providing a high-level interface to run tasks asynchronously using threads (ThreadPoolExecutor) or processes (ProcessPoolExecutor).

Thread pool: പല tasks-ഉം ഏൽപ്പിക്കാവുന്ന, വീണ്ടും വീണ്ടും ഉപയോഗിക്കാവുന്ന worker thread-കളുടെ ഒരു നിശ്ചിത കൂട്ടം; ഒഴിവായ worker കൃവിലെ അടുത്ത task എടുക്കുന്നു.

concurrent.futures: thread-കൾ (ThreadPoolExecutor) അല്ലെങ്കിൽ process-കൾ (ProcessPoolExecutor) ഉപയോഗിച്ച് tasks asynchronous ആയി run ചെയ്യാനുള്ള high-level interface തന്നെ module.

Creating 100 threads for 100 downloads wastes memory and is hard to manage. A pool of, say, 5 workers is created once and reused for all jobs. `executor.submit(f, x)` schedules one call and returns a `Future`; `future.result()` waits for and returns its value. `executor.map(f, items)` runs `f` on every item and returns the results **in input order**. The `with` block waits for all jobs and shuts the pool down automatically.

മലയാളത്തിൽ

ഒരു കാറ്ററിംഗ് സർവീസിൽ ഓരോ പാത്രം കഴുകാനും പുതിയൊരു ജോലിക്കാരനെ വിളിക്കാറില്ല; 5 സ്ഥിരം ജോലിക്കാർ ഉണ്ട്, ഒരാൾ ഒരു പാത്രം കഴുകിക്കഴിഞ്ഞാൽ അടുത്തത് എടുക്കും. അതാണ് thread pool. ആളെ വിളിക്കുന്നതിന്റെയും പറഞ്ഞുവിടുന്നതിന്റെയും ചെലവ് ലാഭം.

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=5) as executor:
    future = executor.submit(func, arg) → future.result()
    results = list(executor.map(func, iterable)) # results in input order
```

EXAMPLE 1

20 jobs of 0.2 s each on a pool of 5 workers (the unit uses 100 jobs of 1 s; the idea is the same).

```
from concurrent.futures import ThreadPoolExecutor
import time

def process_task(task_id):
    time.sleep(0.2)          # pretend to download a file
    return task_id * task_id

start = time.perf_counter()
with ThreadPoolExecutor(max_workers=5) as executor:
    results = list(executor.map(process_task, range(20)))
print("Results:", results[:6], "...", results[-1])
print("Jobs done:", len(results))
print(f"Time: about {time.perf_counter() - start:.1f} s (20 x 0.2 s would be 4 s)")
```

```
Results: [0, 1, 4, 9, 16, 25] ... 361
Jobs done: 20
Time: about 0.8 s (20 x 0.2 s would be 4 s)
```

5 worker-മാർ ഒരേ സമയം 5 ജോലി ചെയ്യുന്നു: $20 \div 5 = 4 \text{ round} \times 0.2 = 0.8$ സെക്കൻഡ്. map result-കൾ input ക്രമത്തിൽ തന്നെ തരുന്നതുകൊണ്ട് output deterministic ആണ്.

EXAMPLE 2

submit() and map() side by side.

```
from concurrent.futures import ThreadPoolExecutor

def square(n):
    return n * n

with ThreadPoolExecutor(max_workers=3) as ex:
    future = ex.submit(square, 7)
    print("submit ->", future.result())
    print("map    ->", list(ex.map(square, [1, 2, 3, 4])))
```

```
submit -> 49
map     -> [1, 4, 9, 16]
```

Exam tip: In the unit's version, each job prints "started"/"completed" from inside the worker, so the printed order varies between runs. Advantages to write: reuse of threads (less creation overhead), a cap on the number of threads, automatic start/join/cleanup, and easy collection of return values.

Thread communication with Queue

queue.Queue: a thread-safe FIFO queue with built-in locking, used to pass data between producer and consumer threads without manual locks.

Producer-consumer problem: the problem of coordinating threads that produce data into a shared buffer with threads that consume it, so that no item is lost or taken from an empty buffer.

queue.Queue: built-in locking ഉള്ള thread-safe FIFO queue; manual lock ഇല്ലാതെ producer, consumer thread-കൾക്കിടയിൽ data കൈമാറാൻ ഉപയോഗിക്കുന്നു.

Producer-consumer problem: ഒരു shared buffer-ലേക്ക് data ഉണ്ടാക്കിയിടുന്ന thread-കളെയും അത് എടുത്ത് ഉപയോഗിക്കുന്ന thread-കളെയും, ഒരു item-ഉം നഷ്ടപ്പെടാതെയും കാലിയായ buffer-ൽ നിന്ന് എടുക്കാതെയും, ഏകോപിപ്പിക്കുന്ന പ്രശ്നം.

`q.put(item)` adds an item; `q.get()` removes one and **blocks while the queue is empty**, so the consumer sleeps without busy waiting. With `Queue(maxsize=n)`, `put()` also blocks while the queue is full. After processing an item the consumer calls `q.task_done()`; `q.join()` blocks until every item put in has been marked done. Because all locking is inside the queue, this is simpler and less error-prone than Lock + Condition.

മലയാളത്തിൽ

ഹോട്ടലിലെ order slip hook പോലെയാണ് Queue. waiter (producer) slip കത്തിയിടുന്നു, cook (consumer) ഒന്നൊന്നായി എടുക്കുന്നു; slip ഇല്ലെങ്കിൽ cook കാത്തിരിക്കുന്നു. ആദ്യം വന്ന order ആദ്യം (FIFO). ആരും തമ്മിൽ തിക്കിത്തിരക്കേണ്ട, lock-ന്റെ ഉത്തരവാദിത്തം hook തന്നെ നോക്കുന്നു.

```
q = Queue(maxsize=0) # 0 = unlimited
q.put(item) # blocks only if the queue is full (maxsize > 0)
item = q.get() # blocks while empty
q.task_done() # one item fully processed
q.join() # wait until all items are processed
```

EXAMPLE

A producer and a daemon consumer running at the same time. Only the consumer prints, and `q.join()` replaces guessing with `sleep()`, so the output is deterministic.

```
from threading import Thread
from queue import Queue
import time

q = Queue()

def producer():
    for i in range(5):
        q.put(f"Item-{i}")
        time.sleep(0.2)

def consumer():
    while True:
        item = q.get()          # blocks while the queue is empty
        print("Consumed:", item)
        q.task_done()

Thread(target=consumer, daemon=True).start()
p = Thread(target=producer)
p.start()
p.join()                      # all 5 items have been put
q.join()                      # all 5 items have been processed
print("All items processed")
```

```
Consumed: Item-0
Consumed: Item-1
Consumed: Item-2
Consumed: Item-3
Consumed: Item-4
All items processed
```

Consumer ഒരു അനന്ത loop ആയതുകൊണ്ട് അതിനെ daemon ആക്കി. എല്ലാ items-ഉം process ആയി എന്ന് `q.join()` ഉറപ്പാക്കിയ ശേഷം main thread തീരുന്നു, daemon consumer തനിയെ നിൽക്കുന്നു. FIFO ആയതുകൊണ്ട് ഇട്ട ക്രമത്തിൽ തന്നെ എടുക്കുന്നു.

Exam tip: The unit's answer key says `q.put()` "blocks if full". That is true only for a bounded queue (`Queue(maxsize=n)` with $n > 0$); the default `Queue()` is unlimited and `put()` never blocks. In the unit's own program the producer prints "Produced" and the consumer prints "Consumed" at nearly the same moment, so that output order may differ between runs.

Timer threads

Timer: a subclass of `Thread` in the `threading` module that runs a given function in a separate thread after a specified delay; it can be stopped with `cancel()` before the delay ends.

Timer: ഒരു നിശ്ചിത delay കഴിഞ്ഞ് തന്നിരിക്കുന്ന function-നെ വേറൊരു thread-ൽ run ചെയ്യുന്ന, `threading` module-ലെ `Thread`-ന്റെ subclass; delay തീരുന്നതിനു മുമ്പ് `cancel()` കൊണ്ട് നിർത്താം.

`Timer(interval, function, args=None, kwargs=None)` creates the timer and `start()` begins the countdown. The main thread is not blocked during the wait, unlike `time.sleep()` in the main program. Uses: reminders, delayed or scheduled actions, retrying a failed request, timeouts.

മലയാളത്തിൽ

അടുപ്പത്ത് കക്കർ വെച്ച് മൊബൈലിൽ 10 മിനിറ്റിന് alarm വെക്കുന്നതുപോലെയാണ് `Timer`. `alarm` അടിക്കുന്നതു വരെ നമുക്ക് മറ്റു പണികൾ ചെയ്യാം (main thread block ആകുന്നില്ല). ഗ്യാസ് നേരത്തെ ഓഫ് ചെയ്താൽ `alarm cancel` ചെയ്യാം (`cancel()`).

EXAMPLE

Run a greeting after 2 seconds, then cancel a second timer.

```
from threading import Timer

def greeting():
    print("Hello after 2 seconds!")

t = Timer(2, greeting)
t.start()
print("Main thread is not blocked")
t.join()

t2 = Timer(5, greeting)
t2.start()
t2.cancel()
print("Second timer cancelled; greeting never runs")
```

```
Main thread is not blocked
Hello after 2 seconds!
Second timer cancelled; greeting never runs
```

"Main thread is not blocked" ആദ്യം വരുന്നു, 2 സെക്കൻഡ് കഴിഞ്ഞ് greeting.

രണ്ടാമത്തെ timer 5 സെക്കൻഡ് തീരുന്നതിനു മുമ്പ് cancel ചെയ്തതുകൊണ്ട് greeting രണ്ടാമത് വരുന്നില്ല.

Multithreading: real-world examples

I/O-bound task: a task whose running time is dominated by waiting for input/output (network, disk, database) rather than by computation; threads speed such tasks up by overlapping the waits.

I/O-bound task: കണക്കുകൂട്ടലിനേക്കാൾ input/output-നായുള്ള (network, disk, database) കാത്തിരിപ്പിലാണ് കൂടുതൽ സമയം പോകുന്ന task; കാത്തിരിപ്പുകൾ ഒന്നിച്ചാക്കി thread-കൾ ഇവയുടെ വേഗം കൂട്ടുന്നു.

The unit closes with four practical patterns: **web scraping** (many network requests at once), a **producer-consumer** restaurant using a Queue, a **background logger** as a daemon thread, and a **bank simulation** protected by a Lock. Each combines tools from the earlier topics.

മലയാളത്തിൽ

ഇതുവരെ പഠിച്ചതെല്ലാം ഇവിടെ ഒരുമിച്ച് ഉപയോഗിക്കുന്നു: ഒരു സമയം പല website-ൽ നിന്ന് data എടുക്കുന്നത്, ഫോട്ടലിലെ order-ഉം cook-ഉം, പശ്ചാത്തലത്തിലെ log എഴുത്ത്, ബാങ്കിലെ ഒരു അക്കൗണ്ടിൽ നിന്നുള്ള ഒരു സമയ പിൻവലിക്കൽ.

EXAMPLE 1: WEB SCRAPING

Three pages that each take 1 s to respond. The network wait is simulated with `sleep()` so the program runs offline; results are printed in sorted order so the output is deterministic.

```
from threading import Thread
import time

pages = {"a.com": 1.0, "b.com": 1.0, "c.com": 1.0}  # URL -> response delay
results = {}

def fetch(url):
    time.sleep(pages[url])          # stands in for waiting on the network
    results[url] = f"<html of {url}>"

start = time.perf_counter()
for url in pages:
    fetch(url)
print(f"Sequential: {time.perf_counter() - start:.0f} s")

results.clear()
start = time.perf_counter()
threads = [Thread(target=fetch, args=(u,)) for u in pages]
for t in threads:
    t.start()
for t in threads:
    t.join()
print(f"Threaded: {time.perf_counter() - start:.0f} s")
for url in sorted(results):
    print(url, "->", results[url])
```

```
Sequential: 3 s
Threaded: 1 s
a.com -> <html of a.com>
b.com -> <html of b.com>
c.com -> <html of c.com>
```

ഒന്നിനു പിറകെ ഒന്നായി 3 സെക്കൻഡ്; thread-കൾ ഉപയോഗിച്ചപ്പോൾ മൂന്ന് കാത്തിരിപ്പും ഒരമിച്ച് നടന്നതുകൊണ്ട് 1 സെക്കൻഡ്. network wait സമയത്ത് GIL release ആകുന്നതുകൊണ്ടാണ് ഇത് സാധ്യമാകുന്നത്. യഥാർത്ഥ program-ൽ `sleep()`-നു പകരം `urllib.request.urlopen(url).read()` പോലുള്ള call വരും.

EXAMPLE 2: PRODUCER-CONSUMER RESTAURANT

A customer places an order every 0.3 s; a chef (daemon) takes 0.5 s per order. The unit's timings are scaled down, and `orders_queue.join()` replaces its `time.sleep(10)`. The order of lines depends on these timings (it was identical over repeated runs here, but could shift on a heavily loaded machine).

```
from threading import Thread
from queue import Queue
import time

orders_queue = Queue()

def customer():
    for n in range(1, 6):
        order = f"Order-{n}"
        print(f"Customer placed {order}")
        orders_queue.put(order)
        time.sleep(0.3)

def chef():
    while True:
        order = orders_queue.get()
        print(f"Chef started {order}")
        time.sleep(0.5)
        print(f"Chef finished {order}")
        orders_queue.task_done()

c = Thread(target=customer)
Thread(target=chef, daemon=True).start()
c.start()
c.join()
orders_queue.join()      # instead of guessing with time.sleep(10)
print("All orders served")
```

```
Customer placed Order-1
Chef started Order-1
Customer placed Order-2
Chef finished Order-1
Chef started Order-2
Customer placed Order-3
Customer placed Order-4
Chef finished Order-2
Chef started Order-3
Customer placed Order-5
Chef finished Order-3
Chef started Order-4
Chef finished Order-4
Chef started Order-5
Chef finished Order-5
All orders served
```

Customer cook-നേക്കാൾ വേഗത്തിൽ order ഇടുമ്പതുകൊണ്ട് order-കൾ queue-ൽ കുന്നുകൂടുന്നു, cook FIFO ക്രമത്തിൽ ഓരോന്നായി തീർക്കുന്നു. എല്ലാ order-ഉം `task_done()` ആയ ശേഷം മാത്രം "All orders served".

EXAMPLE 3: LOGGING IN BACKGROUND

The unit's program: a daemon logger writes every 1 s while the main task runs 3 steps of 2 s. Sample run; the logger and the main task wake at the same instants (0, 2, 4 s), so the order of those lines may differ.

```
import time
from threading import Thread, current_thread

def logger():
    while True:
        print(f"[{current_thread().name}] Writing logs...")
        time.sleep(1)

def main_task():
    for i in range(3):
        print(f"Main task step {i}")
        time.sleep(2)

Thread(target=logger, name="LogDaemon", daemon=True).start()
main_task()
print("Program exiting (daemon log thread stops).")
```

```
[LogDaemon] Writing logs...
Main task step 0
[LogDaemon] Writing logs...
[LogDaemon] Writing logs...
Main task step 1
[LogDaemon] Writing logs...
[LogDaemon] Writing logs...
Main task step 2
[LogDaemon] Writing logs...
[LogDaemon] Writing logs...
Program exiting (daemon log thread stops).
```

6 സെക്കൻഡിനുള്ളിൽ logger ഏകദേശം 6–7 തവണ എഴുതുന്നു. main_task തീർന്ന ഉടനെ program അവസാനിക്കുന്നു; logger daemon ആയതുകൊണ്ട് അതിന്റെ അനന്ത loop തടസ്സമാകുന്നില്ല.

EXAMPLE 4: BANK TRANSACTIONS WITH AND WITHOUT A LOCK

Balance 1000; User1 withdraws 700 and User2 withdraws 400 at the same time. The same code is run once without and once with a lock.

```
from threading import Thread, Lock
import time

def run(use_lock):
    balance = 1000
    lock = Lock()

    def withdraw(name, amount):
        nonlocal balance
        if use_lock:
            lock.acquire()
        try:
            if balance >= amount:
                time.sleep(0.5)          # processing delay
                balance -= amount
                print(f" {name} withdrew {amount}")
            else:
                print(f" {name}: insufficient funds for {amount}")
        finally:
            if use_lock:
                lock.release()

    t1 = Thread(target=withdraw, args=("User1", 700))
    t2 = Thread(target=withdraw, args=("User2", 400))
    t1.start(); time.sleep(0.01); t2.start()
    t1.join(); t2.join()
    print(" Final balance:", balance)

print("Without lock:"); run(False)
print("With lock:");    run(True)
```

```
Without lock:
User1 withdrew 700
User2 withdrew 400
Final balance: -100
With lock:
User1 withdrew 700
User2: insufficient funds for 400
Final balance: 300
```

Lock ഇല്ലാതെ രണ്ടുപേരും 1000 എന്ന balance കണ്ട് check pass ആയി, രണ്ടു withdrawal-ഉം നടന്നു: balance -100, ബാക്കിന് നഷ്ടം. Lock ഉള്ളപ്പോൾ check-ഉം കററിയപ്പോൾ ഒരുമിച്ച് ഒരാൾ മാത്രം ചെയ്യുന്നു: User1 കഴിഞ്ഞ് 300 ബാക്കി, User2-ന്റെ

400 നിരസിക്കപ്പെടുന്നു. check-ഉം update-ഉം ഒരേ lock-നുള്ളിൽ വേണം എന്നതാണ് പാഠം.

Quick recap

- A process has its own memory; a thread is the smallest unit of execution inside a process and shares its memory.
- Thread states: New → Runnable → Running ⇌ Blocked/Waiting → Terminated. A thread can be started only once.
- The GIL lets one thread run Python bytecode at a time: use threads for I/O-bound work, multiprocessing for CPU-bound work.
- Create a thread with `Thread(target=f, args=(...))` or by subclassing `Thread` and overriding `run()`; always call `start()`, not `run()`.
- Default names: "MainThread" and "Thread-N"; `ident` is given by Python at start, `native_id` by the OS; `os.getpid()` gives the process ID.
- `is_alive()`, `main_thread()`, `current_thread()`, `active_count()`, `enumerate()`, `get_native_id()`.
- `t.join()` blocks until `t` ends; start all threads first, then join all. `join()` before `start()` raises `RuntimeError`.
- Daemon threads (`daemon=True`) are killed at program exit; use them only for background work like logging.
- A race condition loses updates when read-modify-write steps of threads interleave; the GIL does not prevent it.
- Lock: one thread at a time; use `with lock:`. Avoid deadlock by acquiring several locks in a fixed order or with timeouts.
- `RLock`: the owning thread may re-acquire it (nested or recursive calls); it keeps a count.
- Condition: `wait()` releases the lock and sleeps, `notify()` wakes a waiter; check the state in a `while` loop.
- `Semaphore(n)`: at most `n` threads inside at once.
- `ThreadPoolExecutor` reuses a fixed set of workers; `queue.Queue` gives thread-safe producer-consumer; `Timer` runs a function after a delay.

Check yourself

Q. 1. What is the primary difference between a process and a thread in terms of memory?

A process has its own independent address space; its data is not directly visible to other processes, which must communicate through pipes, sockets or queues. A thread runs inside a process and shares that process's memory (global variables, heap, code) with the other threads, having only its own stack. So threads communicate easily but need synchronization to avoid data conflicts.

Q. 2. What is the GIL, and how does it affect threading in Python?

The Global Interpreter Lock is a mutex in CPython that lets only one thread execute Python bytecode at a time, even on a multicore CPU. Hence CPU-bound threads get no real speed-up (the countdown example took the same time with two threads). I/O-bound threads still benefit, because the GIL is released while a thread waits for network, disk or database I/O. For CPU-bound work use multiprocessing, where every process has its own GIL.

Q. 3. What happens if you call `join()` before `start()`? What if you call `start()` twice?

Both raise `RuntimeError: "cannot join thread before it is started"` and `"threads can only be started once"`. (The unit's answer key claims `join()` before `start()` returns silently; that is incorrect.)

Q. 4. Describe the thread lifecycle with an example.

New: `t = Thread(target=worker)`. Runnable: after `t.start()`, waiting for the CPU. Running: the scheduler runs `worker()`. Blocked/Waiting: during `time.sleep(1)`, I/O, waiting for a lock or `join()`; it becomes Runnable again when the wait ends. Terminated: `worker()` returns or raises an exception; `t.is_alive()` is then `False` and the thread cannot be restarted. See the program in topic 3.

Q. 5. When should a daemon thread be used? Why is a critical computation never a daemon?

Use a daemon for background services that must not keep the program alive, such as a log writer running while `True`, a monitor or an auto-saver. A daemon is killed abruptly when all non-daemon threads finish, so a critical task (saving data, a payment) run as a daemon could be cut off halfway and its results lost. Such work must be a non-daemon thread, which the program waits for.

Q. 6. What is a race condition? Show how the counter program loses updates and how to fix it.

A race condition is an error where the result depends on the timing of threads accessing shared data without synchronization. In `counter += 1`, thread A reads 5, thread B reads 5, both write 6, so one increment is lost; two threads adding 1000 each gave about 1000 instead of 2000. Fix: put the whole read-modify-write inside `with lock:` using a `threading.Lock`; then the result is always 2000.

Q. 7. What is the difference between Lock and RLock? How do semaphores differ from locks?

A Lock can be held once; if the same thread tries to acquire it again it blocks forever. An RLock records its owner and a count, so the owning thread can acquire it repeatedly (nested or recursive calls) and it is free only after the same number of releases. A Lock allows one thread in the critical section (one key); a Semaphore(n) keeps a counter and allows up to n threads at once (n keys), blocking others when the counter reaches 0.

Q. 8. Explain the producer-consumer problem and how Condition variables solve it without wasting CPU time.

Producers put items into a shared buffer and consumers take them out. Without coordination a consumer may take from an empty buffer, or a producer may add to a full one. With a Condition, both work inside with cond:. The consumer runs while not buffer: cond.wait(); wait() releases the lock and sleeps, so no CPU is used. The producer adds an item and calls cond.notify(); the consumer wakes, re-acquires the lock, re-checks the buffer and takes the item. For a bounded buffer the producer similarly waits while the buffer is full and the consumer notifies after removing an item.

Q. 9. What causes deadlock and starvation? How can deadlock be avoided when using several locks?

Deadlock: two or more threads each hold a lock and wait for a lock held by another, so none proceeds (for example, T1 holds A and wants B while T2 holds B and wants A). Starvation: a thread keeps waiting because others are always scheduled or given the lock first; the program still progresses, but that thread does not. Avoid deadlock by (1) acquiring locks in one fixed global order, (2) using acquire(timeout=...) and backing off on failure, (3) holding fewer locks or using a queue.Queue.

Q. 10. Why use a thread pool and a Queue instead of manual threads and locks? Why does threading help web scraping?

A ThreadPoolExecutor reuses a fixed number of workers, so 100 downloads do not create 100 threads, and it handles starting, joining, cleanup and return values (20 jobs of 0.2 s took 0.8 s on 5 workers). A queue.Queue has locking built in: get() blocks while empty and task_done()/join() track completion, so there are no manual wait()/notify() calls to get wrong. Web scraping is I/O-bound: while one thread waits for a server, others send their requests, and the GIL is released during network waits, so 3 pages of 1 s each took 1 s instead of 3 s.

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.